Teoría de Algoritmos
(TB024/75.29) Curso Buchwald - Genender

Trabajo Práctico 3
Problemas NP-Completos para la defensa de la Tribu del Agua

23 de noviembre de 2025

Facundo Camilo Stifman
112541

Juan Ignacio Moore
112479

Avril Victoria Morfeo Zerbi
112563

Martin Ariel Kupa
112560

Índice

1. Demostración NP-Completo **3**
1.1. 2-Partition . . . 3
1.1.1. Equivalencia a *Subset Sum* . . . 3
1.2. Tribu del Agua . . . 4
1.2.1. Reducción desde *2-Partition* . . . 4

2. Backtracking **6**
2.1. Podas . . . 6
2.2. Implementación . . . 6

3. Programación Lineal **9**
3.1. Enunciado del Problema y Desafío No Lineal . . . 9
3.2. Solución Óptima (Linearización del Modelo) . . . 9
3.2.1. Variables de Decisión . . . 9
3.2.2. Linearización de la Función Objetivo . . . 9
3.2.3. Modelo Completo (4.a) . . . 11
3.2.4. Implementación . . . 11

4. Aproximación **13**
4.1. Descripción del algoritmo . . . 13
4.1.1. Implementación . . . 14
4.2. Análisis de complejidad temporal . . . 14
4.2.1. Fase de ordenamiento . . . 14
4.2.2. Fase de asignación . . . 14
4.2.3. Complejidad total . . . 14
4.3. Evaluación de la calidad de la aproximación . . . 15
4.4. Conclusión . . . 17

5. Mediciones **17**
5.1. Backtracking . . . 18
5.2. Programación Líneal . . . 19
5.3. Aproximación . . . 20
5.4. Conclusiones . . . 21

A. Resultados sobre sets de la catedra **22**

B. Resultados de las mediciones **25**

C. Código de generación de datos **26**

1. Demostración NP-Completo

1.1. 2-Partition

Para la demostración de que *Tribu del Agua* es un problema NP-Completo utilizaremos el problema de *2-Partition*:

> Dada una lista A de n números enteros, se desea saber si es posible particionar la lista en dos sublistas S_1 y S_2 tales que $A = S_1 \cup S_2$ y $S_1 \cap S_2 = \emptyset$, cumpliendo $\sum_{x \in S_1} x = \sum_{x \in S_2} x$.

Dado que este problema no fue probado en clase, procedemos a dar una pequeña demostración utilizando *Subset Sum*.

1.1.1. Equivalencia a *Subset Sum*

El problema de *2-Particion* es equivalente a una instancia particular de *Subset Sum* donde el valor objetivo es la mitad de la sumatoria total de la lista.

Una instancia arbitraria de *Subset Sum*, con lista A de sumatoria $t := \sum_{x \in A} x$ y objetivo o, se puede transformar a dicha instancia particular definiendo una nueva lista $A' = A \cup \{t - 2o\}$ y un nuevo objetivo $o' = t - o$. Nótese que la sumatoria de la nueva lista cumple la igualdad que esperamos $\sum_{x \in A'} x = 2\,(t - o) = 2o'$.

Si hay una solución $S' \subseteq A'$, tanto S' como $A' \setminus S'$ tienen sumatoria $t - o$ y alguno de estos conjuntos contiene el elemento adicional $t - 2o$. Si eliminamos dicho elemento obtenemos un nuevo conjunto cuya sumatoria es $o' - (t - 2o) = o$, por lo que obtuvimos también una solución a la instancia original del problema.

Si hay una solución al problema original $S \subset A$, se puede crear otro conjunto $S' = S \cup \{t - 2o\}$ el cual cumple con $S' \subseteq A'$ y $\sum_{x \in S'} x = o + t - 2o = t - o = o'$, por lo que obtuvimos una solución de la instancia particular.

Por último, para probar que *2-Partition* está en NP basta con comprobar que el siguiente verificador se ejecuta en $O(n^2)$ (nótese que la complejidad puede mejorarse si se utilizan multisets con búsqueda $O(1)$ en lugar de listas):

```python
def es_dos_particion(lista: list, sublista1: list, sublista2: list, suma_objetivo):
    if len(sublista1) + len(sublista2) != len(lista):
        return False

    suma_sublista1 = 0
    for numero in sublista1:
        if numero not in lista:
            return False
        suma_sublista1 += numero

    suma_sublista2 = 0
    for numero in sublista2:
        if numero not in lista:
            return False
        suma_sublista2 += numero

    for numero in lista:
        if numero not in sublista1 and numero not in sublista2:
            return False

    return suma_objetivo == suma_sublista1 == suma_sublista2
```

Finalmente, hemos probado que *2-Partition* es un problema NP-Completo.

1.2. Tribu del Agua

Primero demostraremos que el problema de decisión de la Tribu del Agua, en adelante TA, está en NP.

Considere el siguiente verificador:

```python
def es_solucion_TA(lista_maestros: list, sublistas: list[list], cota_superior):
    if len(sublistas) > len(lista_maestros):
        return False
    maestros_vistos = set()
    habilidades_por_grupo = [0] * len(sublistas)
    for i in range(len(sublistas)):
        grupo = sublistas[i]
        if len(grupo) > len(lista_maestros):
            return False
        for maestro in grupo:
            if maestro not in lista_maestros:
                return False
            if maestro in maestros_vistos:
                return False
            maestros_vistos.add(maestro)
            habilidades_por_grupo[i] += maestro.habilidad

    for maestro in lista_maestros:
        if maestro not in maestros_vistos:
            return False

    resultado = reduce(lambda acum, x: acum + x**2, habilidades_por_grupo, 0)
    return resultado <= cota_superior
```

El verificador propuesto recorre los maestros de cada grupo (trabajo n), comprobando si están presentes en la lista original (trabajo n si se recibe una lista, pero se podría mejorar utilizando un multiset) y sumando sus habilidades. Luego comprueba si los grupos contienen a todos los maestros de la lista (trabajo n) y por último comprueba que la sumatoria del cuadrado de sus habilidades no supere la cota requerida. El trabajo total es $O(n^2)$, por lo que este verificador es eficiente.

1.2.1. Reducción desde *2-Partition*

A continuación probaremos que 2P $\leq_p$ TA. Considere una instancia arbitraria de 2P, donde recibimos una lista A de n números y un valor objetivo o. Buscamos particionar A en dos sublistas de manera que la sumatoria de elementos de cada sublista sea o. Como las sublistas deben formar una partición de la lista original todos los elementos de A deben estar en alguna de ellas, de manera que $s_1 + s_2 = \sum_{x \in A} x = t$. La única forma de que ambas sublistas sumen lo mismo es si $o = s_1 = s_2 = \frac{t}{2}$. Si el valor objetivo no es la mitad de la sumatoria total de elementos de A, no existe la *2-partición*. En los casos donde $o = \frac{t}{2}$, podemos resolver el problema mediante TA. Creamos un maestro m_i por cada elemento $x_i \in A$, de manera que la habilidad del maestro coincida con el valor x_i. La cota de habilidad que pediremos en TA sera $H = \frac{t^2}{2}$ y particionaremos en dos grupos $k = 2$.

Es trivial notar que, si existe 2P, tambien existe TA; basta con construir los grupos M_1 y M_2 utilizando los maestros correspondientes a los elementos de la solución de 2P. Como la habilidad de los maestros se corresponde con los valores de 2P, tenemos que $o = \frac{t}{2} = \sum_{x \in M_1} x = \sum_{x \in M_2} x$. Reemplazando en la condición de TA obtenemos:

$$\sum_{i=1}^{k}\left(\sum_{x\in S_i} x\right)^2 \leq H$$
$$\sum_{i=1}^{2}\frac{t^2}{4} \leq H$$
$$\frac{t^2}{2} \leq H$$
$$H \leq H$$

Por otro lado, si existe TA tenemos dos subgrupos G_1 y G_2 tales que:

$$G_1 + G_2 = \sum_i m_i = \sum_{x\in A} x = t \tag{1}$$
$$(G_1)^2 + (G_2)^2 \leq \frac{t^2}{2} \tag{2}$$

Reemplazando t obtenemos:

$$(G_1)^2 + (G_2)^2 \leq \frac{(G_1+G_2)^2}{2}$$
$$(G_1)^2 + (G_2)^2 \leq \frac{(G_1)^2}{2} + G_1G_2 + \frac{(G_2)^2}{2}$$
$$\frac{(G_1)^2}{2} + \frac{(G_2)^2}{2} \leq G_1G_2$$
$$\frac{(G_1)^2}{2} - G_1G_2 + \frac{(G_2)^2}{2} \leq 0$$
$$\left(\frac{G_1}{\sqrt{2}} - \frac{G_2}{\sqrt{2}}\right)^2 \leq 0$$

Nótese que $x^2 = 0$ si y solo si $x = 0$, por lo que:

$$\frac{G_1}{\sqrt{2}} - \frac{G_2}{\sqrt{2}} = 0$$
$$\frac{1}{\sqrt{2}}(G_1 - G_2) = 0$$
$$G_1 = G_2$$

Si reemplazamos en (1) con $G = G_1 = G_2$ tenemos:

$$G_1 + G_2 = t$$
$$2G = t$$
$$G = \frac{t}{2}$$

Finalmente, los grupos creados por TA cumplen con las condiciones de 2P, por lo que una solución de TA implica una solución de 2P.

Con esto hemos demostrado que:

$$\forall x \in NP,\ x \leq_p SS \leq_p 2P \leq_p TA$$

Por transitividad obtenemos $\forall x \in NP$, $x \leq_p$ TA, cumpliendo con la segunda condición de los problemas NP-Completos. Dado que ya probamos que TA $\in$ NP, concluimos que TA es NP-Completo.

2. Backtracking

El algoritmo de backtracking recorre el espacio de soluciónes construyendo, de forma incremental, una asignación de maestros a grupos. En cada paso se toma al siguiente maestro de la lista y se lo intenta ubicar en cada uno de los k grupos posibles. Para cada elección, se actualizan la suma de fuerzas del grupo correspondiente y el coeficiente parcial de la asignación actual.

Este proceso continúa recursivamente hasta que todos los maestros fueron asignados, momento en el cual se calcula el coeficiente total de la partición resultante.

2.1. Podas

A lo largo de la exploración, se conserva el mejor coeficiente encontrado y la asignación asociada. Cuando una decisión parcial demuestra que no puede conducir a una solución mejor que la actual, la rama se descarta y se retrocede para probar otras alternativas.

Inicialmente se ordenaron los maestros por fuerza descendente para facilitar un reparto más equilibrado de los grupos desde el inicio. Esto permite aplicar podas que evitan evaluar combinaciones que no pueden mejorar la solución. Las podas implementadas fueron:

- **Poda por mejor caso:** Dada la solución parcial actual y la suma de habilidades de maestros aun no asignados se estima un cota inferior del coeficiente final. Dicha estimacion se calcula suponiendo que la habilidad restante pudiera fraccionarse y se distribuyese de forma perfectamente equitativa entre los grupos de menor habilidad. Si este valor estimado es mayor o igual al mejor coeficiente registrado hasta el momento, se descarta la rama.
- **Poda por simetría de grupos:** El algoritmo mantiene una lista con la suma de habilidades de cada grupo de la solución parcial. Si luego de asignar un maestro a un grupo encontramos que otro grupo tiene la misma suma de habilidades, se omite la asignación del maestro. Esto debido a que dicha rama produciría un coeficiente equivalente al previamente evaluado, difiriendo unicamente en la permuta del maestro.
- **Poda por aproximación greedy:** Previo a la ejecución del algoritmo por backtracking se ejecuta un algoritmo greedy para aproximar una solución que minimize el coeficiente objetivo. Dicho algoritmo asigna el maestro de mayor habilidad al grupo de menor suma, en un intento de distribuir las habilidad de forma equitativa. La solución obtenida mediante este metodo se utiliza como heurística para podar casos en el algoritmo de backtracking.

2.2. Implementación

Su implementación en Python es la siguiente:

```python
def calcular_mejor_caso(habilidad_total, habilidad_restante, sumas, orden_sumas):
    """
    Calcula el coeficiente minimo que se obtendria a partir de la solucion parcial,
    recibida mediante `sumas` y `habilidad_restante`, suponiendo que la habilidad de
    los maestros aun no asignados se pudiera fraccionar.

    `orden_sumas` es una lista de los indices de `sumas` en orden de menor a mayor
    """
    # Obtiene la habilidad del i-esimo grupo con mayor habilidad. Dado que orden_sumas
    # esta en orden ascendente, hay que invertir el acceso.
    obtener_habilidad_grupo = lambda i: sumas[orden_sumas[len(sumas)-1-i]]
    cantidad_grupos = len(sumas)

    # La habilidad total de los maestros presentes en los grupos como estan en la ↩
        solucion parcial.
    habilidad_ocupada = habilidad_total - habilidad_restante

    # La habilidad de los grupos que se dejan como estan
    habilidad_ignorada = 0
```

```
    coeficiente_ignorado = 0

    mejor_caso = 0
    for i in range(cantidad_grupos):
        # Buscamos igualar la habilidad de todos los grupos a la del mayor grupo que podamos.
        habilidad_objetivo = obtener_habilidad_grupo(i)

        if i > 0:
            habilidad_ignorada += obtener_habilidad_grupo(i-1)
            coeficiente_ignorado += obtener_habilidad_grupo(i-1)**2

        nueva_habilidad_ocupada = habilidad_ignorada + habilidad_objetivo*(cantidad_grupos - i)
        habilidad_utilizada = nueva_habilidad_ocupada - habilidad_ocupada

        if habilidad_utilizada > habilidad_restante:
            # La habilidad a asignar no alcanza para igualar las habilidades de los grupos
            # menores al objetivo. Reintentar con un grupo mas chico.
            continue

        # Si tras igualar la mayor cantidad de grupos posibles aun nos queda habilidad
        # a asignar, la distribuiremos equitativamente entre los grupos.
        habilidad_excedente = habilidad_restante - habilidad_utilizada
        habilidad_extra_por_grupo = habilidad_excedente/(cantidad_grupos-i)
        mejor_caso = coeficiente_ignorado + (cantidad_grupos - i)*(habilidad_objetivo + habilidad_extra_por_grupo)**2
        break

    return mejor_caso

def conseguir_grupos(
    maestros,
    grupos_actuales,
    sumas,
    habilidad_total,
    habilidad_restante,
    coef_record,
    coef_parcial,
    i,
) -> tuple[list, int]:
    cant_grupos = len(grupos_actuales)
    if i == len(maestros):
        return [g.copy() for g in grupos_actuales], coef_parcial

    # Ordenar indices de grupos por suma ascendente (llenar primero los mas vacios)
    indices_ordenados = sorted(range(cant_grupos), key=lambda j: sumas[j])
    mejor_caso = calcular_mejor_caso(habilidad_total, habilidad_restante, sumas, indices_ordenados)
    if mejor_caso >= coef_record:
        return None, coef_record

    nom, fuerza = maestros[i]
    mejor_sol = None
    vistos = set()

    for j in indices_ordenados:
        # Si este grupo tiene la misma suma que el anterior daria el mismo resultado
        if sumas[j] in vistos:
            continue
        vistos.add(sumas[j])

        # Poda 3: Descarto si el coeficiente parcial ya supera el record
        nuevo_parcial = coef_parcial - sumas[j] ** 2 + (sumas[j] + fuerza) ** 2
        if nuevo_parcial >= coef_record:
            continue

        grupos_actuales[j].append(nom)
        sumas[j] += fuerza
```

```python

        sol, coef = conseguir_grupos(
            maestros,
            grupos_actuales,
            sumas,
            habilidad_total,
            habilidad_restante - fuerza,
            coef_record,
            nuevo_parcial,
            i + 1,
        )
        if sol is not None and coef < coef_record:
            coef_record = coef
            mejor_sol = sol

        grupos_actuales[j].pop()
        sumas[j] -= fuerza

    return mejor_sol, coef_record


def heuristica_greedy(maestros, cant_grupos):
    """Genera una solucion inicial usando heuristica greedy para mejorar la poda"""
    grupos = [[] for _ in range(cant_grupos)]
    sumas = [0 for _ in range(cant_grupos)]

    # Asignar cada maestro al grupo con menor suma actual
    for nom, fuerza in maestros:
        idx_min = min(range(cant_grupos), key=lambda i: sumas[i])
        grupos[idx_min].append(nom)
        sumas[idx_min] += fuerza

    coef = sum(s**2 for s in sumas)
    return grupos, coef, sumas

def grupos_maestros_bt(lista_maestros, cant_grupos) -> tuple[list, int]:
    if cant_grupos <= 0:
        return [], 0
    elif cant_grupos == 1:
        return [[nom for nom, _ in lista_maestros]], sum(
            s**2 for _, s in lista_maestros
        )
    elif cant_grupos >= len(lista_maestros):
        grupos = [[nom] for nom, _ in lista_maestros]
        grupos += [
            [] for _ in range(cant_grupos - len(lista_maestros))
        ]  # Los sobrantes los devuelvo vacios
        return grupos, sum(s * s for _, s in lista_maestros)

    ordenados = sorted(lista_maestros, key=lambda x: x[1], reverse=True)
    grupos_actuales = [[] for _ in range(cant_grupos)]
    sumas = [0 for _ in range(cant_grupos)]
    total = sum(m for _, m in ordenados)

    # Obtener solucion inicial con heuristica greedy para mejorar la poda
    grupos_greedy, coef_greedy, _ = heuristica_greedy(ordenados, cant_grupos)

    grupos, coef = conseguir_grupos(
        ordenados,
        grupos_actuales,
        sumas,
        habilidad_total=total,
        habilidad_restante=total,
        coef_record=coef_greedy,
        coef_parcial=0,
        i=0
    )

    # Si el backtracking no encuentra nada mejor (todas las ramas fueron podadas),
    # devolver la solucion greedy
```

```
    if grupos is None:
        return grupos_greedy, coef_greedy

    return grupos, coef
```

Las podas implementadas reducen significativamente la cantidad de combinaciones que se evalúan en la práctica, pero no modifican la cota teórica en el peor caso. El algoritmo debera evaluar que sucedería si se asigna un maestro a cada uno de los k grupos y este proceso se repite por los n maestros. Por lo tanto, la complejidad total del algoritmo resulta $O(k^n)$.

3. Programación Lineal

Para complementar la solución exacta obtenida mediante Backtracking, en esta sección se explora el uso de la Programación Lineal para modelar y resolver el "Problema de la Tribu del Agua".

3.1. Enunciado del Problema y Desafío No Lineal

El problema de optimización original, planteado por el Maestro Pakku, busca minimizar la adición de los cuadrados de las sumas de las fuerzas de los k grupos $(S_1, \ldots, S_k)$:

$$\text{mín} \sum_{i=1}^{k} \left(\sum_{x_j \in S_i} x_j \right)^2$$

Esto implica un desafío computacional, por la función objetivo cuadrática. Los métodos de Programación Lineal puros están diseñados para optimizar funciones objetivo lineales. La consigna proponía dos alternativas (4.a y 4.b).

En esta sección nos enfocaremos en desarrollar el punto 4.a: obtener la solución óptima al problema original, lo cual, como advertía el enunciado, implica agregar bastantes variables de decisión para lograr una linearización.

3.2. Solución Óptima (Linearización del Modelo)

Para obtener la solución óptima sin recurrir a un *solver* que soporte Programación Cuadrática, optamos por una estrategia de linearización. Este enfoque requiere la introducción de un número significativo de variables auxiliares para manejar los términos cuadráticos.

3.2.1. Variables de Decisión

- Sea n el número de maestros y k el número de grupos.
- Sea h_j la habilidad (fuerza) del maestro j.
- Se define una variable binaria z_{ij}:

$$z_{ij} = \begin{cases} 1 & \text{si el maestro } j \text{ es asignado al grupo } i \\ 0 & \text{en caso contrario} \end{cases}$$

3.2.2. Linearización de la Función Objetivo

La suma de habilidades del grupo i es $S_i = \sum_{j=1}^{n} z_{ij} h_j$. La función objetivo es $\text{mín}\, Z = \sum_{i=1}^{k} (S_i)^2$. Primero, expandimos el término cuadrático $(S_i)^2$. Al elevar una sumatoria al cuadrado,

obtenemos la suma de los términos individuales al cuadrado, más el doble de todos los productos cruzados:

$$(S_i)^2 = \left(\sum_{j=1}^{n} z_{ij} h_j\right)^2 = \sum_{j=1}^{n} (z_{ij})^2 (h_j)^2 + \sum_{j \neq l} (z_{ij} z_{il})(h_j h_l)$$

Donde l es, al igual que j, un índice que recorre los maestros de 1 a n. La notación $\sum_{j \neq l}$ es una forma de abreviar la suma sobre todos los pares de maestros (j, l) donde j y l son distintos. Dado que z_{ij} es binaria, $z_{ij}^2 = z_{ij}$. Separando la segunda suma (para $j < l$):

$$(S_i)^2 = \sum_{j=1}^{n} z_{ij} (h_j)^2 + \sum_{j<l} 2(z_{ij} z_{il})(h_j h_l)$$

Sustituimos esto en la función objetivo Z:

$$Z = \sum_{i=1}^{k} \left(\sum_{j=1}^{n} z_{ij} (h_j)^2 + \sum_{j<l} 2(h_j h_l)(z_{ij} z_{il}) \right)$$

Reordenando los términos:

$$Z = \sum_{j=1}^{n} (h_j)^2 \left(\sum_{i=1}^{k} z_{ij} \right) + \sum_{i=1}^{k} \sum_{j<l} 2(h_j h_l)(z_{ij} z_{il})$$

La primera restricción del problema es que cada maestro j debe estar en exactamente un grupo, es decir, $\sum_{i=1}^{k} z_{ij} = 1$ para todo j. Sustituyendo esto, el primer término $\sum_{j=1}^{n} (h_j)^2 \cdot (1)$ se convierte en una constante (C) que solo depende de los datos de entrada.

Dado que C es constante, minimizar $Z = C + \text{resto}$ es equivalente a minimizar solo el resto. Por lo tanto, nuestro problema se reduce a minimizar un nuevo objetivo Z':

$$\text{mín}\, Z' = \sum_{i=1}^{k} \sum_{j<l} 2(h_j h_l)(z_{ij} z_{il})$$

Este objetivo sigue siendo cuadrático debido al producto $z_{ij} z_{il}$. Para linearizarlo, introducimos las variables de decisión como advertía la consigna.

El truco consiste en reemplazar el producto cuadrático $(z_{ij} \cdot z_{il})$ por una nueva variable única, u_{ijl}. Esta nueva variable debe *representar* la decisión conjunta:

- Se define una variable continua u_{ijl} (por cada i, j, l con $j < l$) que forzaremos (mediante restricciones) a que tome el valor 1 si y solo si los maestros j y l están **ambos** asignados al grupo i, y 0 en cualquier otro caso.

Es decir, esta variable u_{ijl} captura el valor de la expresión ($z_{ij} = 1$ Y $z_{il} = 1$).

Al realizar este reemplazo, la función objetivo se vuelve **lineal**:

$$\text{mín}\, Z' = \sum_{i=1}^{k} \sum_{j=1}^{n} \sum_{l=j+1}^{n} (2 \cdot h_j \cdot h_l) \cdot u_{ijl}$$

Esta es la función objetivo implementada en el código. Para asegurar que u_{ijl} tome el valor del producto, añadimos las siguientes restricciones de linearización para cada i, j, l con $j < l$:

1. $u_{ijl} \leq z_{ij}$
2. $u_{ijl} \leq z_{il}$
3. $u_{ijl} \geq z_{ij} + z_{il} - 1$

3.2.3. Modelo Completo (4.a)

El modelo final de Programación Lineal es:

Variables:

- $z_{ij} \in \{0,1\} \quad \forall i \in \{1..k\}, j \in \{1..n\}$
- $u_{ijl} \in [0,1] \quad \forall i \in \{1..k\}, 1 \leq j < l \leq n$

Función Objetivo (Lineal):

$$\text{mín} \sum_{i=1}^{k} \sum_{j=1}^{n} \sum_{l=j+1}^{n} (2 \cdot h_j \cdot h_l) \cdot u_{ijl}$$

Sujeto a:

1. Asignación Única: Cada maestro debe ser asignado a un solo grupo.

$$\sum_{i=1}^{k} z_{ij} = 1 \quad \forall j \in \{1, \ldots, n\}$$

2. Restricciones de Linearización: Para todo i, j, l con $j < l$.

$$u_{ijl} \leq z_{ij}$$
$$u_{ijl} \leq z_{il}$$
$$u_{ijl} \geq z_{ij} + z_{il} - 1$$

Este modelo posee $O(n \cdot k)$ variables binarias y $O(k \cdot n^2)$ variables continuas (y restricciones), lo que explica el alto costo computacional para resolverlo.

3.2.4. Implementación

El modelo 4.a fue implementado utilizando la biblioteca `PuLP` en Python.

Implementación del modelo linearizado (4.a) en PuLP.

```python
def grupos_maestros_pl(
    values: List[float],
    k: int,
    time_limit: Optional[float] = None,
    verbose: bool = False,
) -> Tuple[float, List[List[int]]]:
    """
    Formulacion MILP exacta (Modelo A) para minimizar sum_{i=1..k} (sum_{j in S_i} values[j])^2
    usando variables binarias z_{i,j} (asignacion de elemento j al grupo i) y variables continuas
    u_{i,j,l} para representar z_{i,j} * z_{i,l} (solo j < l). El termino sum_j values[j]^2 es constante
    y se omite del objetivo; el objetivo de la MILP es minimizar
        sum_{i=1..k} sum_{1<=j<l<=n} 2 * values[j] * values[l] * u_{i,j,l}

    Parametros:
      - values: lista de valores (positivos)
      - k: numero de grupos
      - time_limit: limite de tiempo (segundos) para el solver (opcional)
      - verbose: si True muestra mensajes del solver

    Devuelve (sum_squares, groups) donde sum_squares es el valor exacto de la suma de cuadrados
```

```
    calculado en Python a partir de la particion encontrada, y groups es una lista de k listas con
    los indices originales de los elementos asignados a cada grupo.
    """
    n = len(values)
    if k <= 0:
        raise ValueError("k debe ser >= 1")
    if n == 0:
        return 0.0, [[] for _ in range(k)]

    prob = pulp.LpProblem("Tribu_Del_Agua", pulp.LpMinimize)

    # variables binarias z_{i,j}
    z: Dict[Tuple[int, int], pulp.LpVariable] = {}
    for i in range(k):
        for j in range(n):
            z[(i, j)] = pulp.LpVariable(f"z_{i}_{j}", cat="Binary")

    # variables continuas u_{i,j,l} para j < l
    u: Dict[Tuple[int, int, int], pulp.LpVariable] = {}
    for i in range(k):
        for j in range(n):
            for l in range(j + 1, n):
                u[(i, j, l)] = pulp.LpVariable(
                    f"u_{i}_{j}_{l}", lowBound=0, upBound=1, cat="Continuous"
                )

    # (Opcional) variables S_i para reporte
    S: Dict[int, pulp.LpVariable] = {
        i: pulp.LpVariable(f"S_{i}", lowBound=0, cat="Continuous") for i in range(k)
    }

    # Objetivo: sum_i sum_{j<l} 2 * values[j] * values[l] * u_{i,j,l}
    obj_terms = []
    for i in range(k):
        for j in range(n):
            for l in range(j + 1, n):
                coef = 2.0 * values[j] * values[l]
                obj_terms.append(coef * u[(i, j, l)])
    prob += pulp.lpSum(obj_terms)

    # Restriccion: cada elemento asignado a exactamente un grupo
    for j in range(n):
        prob += pulp.lpSum(z[(i, j)] for i in range(k)) == 1, f"assign_item_{j}"

    # Definicion S_i = sum_j values[j] * z_{i,j}
    for i in range(k):
        prob += (
            S[i] == pulp.lpSum(values[j] * z[(i, j)] for j in range(n)),
            f"define_S_{i}",
        )

    # Linearizacion u_{i,j,l} = z_{i,j} * z_{i,l}  (j < l)
    for i in range(k):
        for j in range(n):
            for l in range(j + 1, n):
                zp = z[(i, j)]
                zq = z[(i, l)]
                upq = u[(i, j, l)]
                prob += upq <= zp, f"lin1_u_{i}_{j}_{l}"
                prob += upq <= zq, f"lin2_u_{i}_{j}_{l}"
                prob += upq >= zp + zq - 1, f"lin3_u_{i}_{j}_{l}"

    # prob += z[(0, 0)] == 1

    # Seleccionar solver CBC con time limit si se proporciona
    solver = pulp.PULP_CBC_CMD(msg=1 if verbose else 0, timeLimit=time_limit)

    start = time.perf_counter()
    prob.solve(solver)
```

```python
    elapsed = time.perf_counter() - start

    # Reconstruir grupos segun z
    groups: List[List[int]] = [[] for _ in range(k)]
    unassigned = []
    for j in range(n):
        assigned = False
        for i in range(k):
            val = pulp.value(z[(i, j)])
            if val is not None and val > 0.5:
                groups[i].append(j)
                assigned = True
                break
        if not assigned:
            unassigned.append(j)

    # Calcular suma de cuadrados real en Python
    sum_squares = 0.0
    for g in groups:
        s = sum(values[j] for j in g)
        sum_squares += s * s

    # ordenar indices dentro de cada grupo
    for g in groups:
        g.sort()

    # Informacion opcional
    if verbose:
        obj_val = pulp.value(prob.objective)
        try:
            mip_status = pulp.LpStatus[prob.status]
        except Exception:
            mip_status = str(prob.status)
        print(
            f"Solver status: {mip_status}, MILP objective (without const): {obj_val}, ↩
                time={elapsed:.3f}s"
        )

    return sum_squares, groups, unassigned
```

4. Aproximación

4.1. Descripción del algoritmo

Dado que el problema de asignación de maestros es NP-Completo, resulta fundamental desarrollar algoritmos de aproximación que permitan obtener solucionós de buena calidad en tiempo polinomial. El algoritmo propuesto se basa en una estrategia greedy que minimiza localmente el incremento de la función objetivo.

El algoritmo opera en las siguientes etapas:

1. Inicialización de k grupos vacíos: $G_1, G_2, \ldots, G_k$, con sumas iniciales $s_1 = s_2 = \cdots = s_k = 0$;
2. Ordenamiento de los maestros $(m_1, f_1), (m_2, f_2), \ldots, (m_n, f_n)$ en orden decreciente de fuerza, tal que $f_1 \geq f_2 \geq \cdots \geq f_n$;
3. Para cada maestro m_j con fuerza f_j, asignar al grupo G_i que minimice el incremento cuadrático:
$$i^* = \underset{i \in \{1,\ldots,k\}}{\arg\min} \left[(s_i + f_j)^2 - s_i^2\right]$$
4. Actualizar la suma del grupo seleccionado: $s_{i^*} \leftarrow s_{i^*} + f_j$;
5. Repetir el proceso hasta asignar todos los maestros.

4.1.1. Implementación

```python
def grupos_maestros_aprox(lista_maestros, cant_grupos) -> tuple[list, int]:
    ordenados = sorted(lista_maestros, key=lambda x: x[1], reverse=True)
    grupos = [[] for _ in range(cant_grupos)]
    sumas = [0 for _ in range(cant_grupos)]

    for nom, fuerza in ordenados:
        indice = min(range(cant_grupos), key=lambda i: (sumas[i] + fuerza)**2 - sumas[i]**2)
        grupos[indice].append(nom)
        sumas[indice] += fuerza

    return grupos, sum(s**2 for s in sumas)
```

4.2. Análisis de complejidad temporal

El análisis de complejidad del algoritmo se descompone en las siguientes fases:

4.2.1. Fase de ordenamiento

El ordenamiento inicial de los n maestros por fuerza requiere una complejidad de:

$$T_{\text{sort}}(n) = O(n \log n)$$

Esta operación se realiza una única vez al inicio del algoritmo, utilizando algoritmos de ordenamiento eficientes como *Timsort* (implementado por defecto en Python).

4.2.2. Fase de asignación

En cada iteración del bucle principal, el algoritmo debe:

- Evaluar el incremento cuadrático $(s_i + f_j)^2 - s_i^2$ para cada uno de los k grupos;
- Seleccionar el grupo que minimiza dicho incremento;
- Actualizar la suma del grupo seleccionado.

Para un maestro dado con fuerza f_j, el cálculo del incremento en el grupo i se expande como:

$$(s_i + f_j)^2 - s_i^2 = s_i^2 + 2s_i f_j + f_j^2 - s_i^2 = 2s_i f_j + f_j^2$$

Dado que f_j es constante para todos los grupos en una iteración, minimizar $2s_i f_j + f_j^2$ equivale a minimizar s_i. Sin embargo, el algoritmo explícitamente evalúa el incremento cuadrático completo para mayor claridad conceptual.

Cada iteración requiere inspeccionar los k grupos, y el proceso se repite para los n maestros, resultando en:

$$T_{\text{assign}}(n, k) = O(n \cdot k)$$

4.2.3. Complejidad total

La complejidad temporal total del algoritmo es:

$$T(n, k) = T_{\text{sort}}(n) + T_{\text{assign}}(n, k) = O(n \log n) + O(n \cdot k)$$

En la práctica, cuando $k > \log n$ (que es el caso común en las instancias de interés), el término dominante es $O(n \cdot k)$, por lo que:

$$T(n, k) = O(n \cdot k)$$

En el peor caso, cuando $k = n$ (cada maestro en un grupo separado), la complejidad temporal es:

$$T(n) = O(n \log n) + O(n^2) = O(n^2)$$

La complejidad espacial del algoritmo es $O(n + k)$, correspondiente al almacenamiento de los maestros ordenados y las estructuras de datos de los grupos.

4.3. Evaluación de la calidad de la aproximación

Para evaluar la calidad de la aproximación se comparan sus resultados con los obtenidos mediante el algoritmo de backtracking y se calcula la proporción por la que la aproximación excede la solución optima.

```
==== Set 5_2.txt ====
    Resultado Backtracking: 1894340
    Resultado Aproximacion: 1894340
    Diferencia: 0
    Proporcion: 1.0

==== Set 6_3.txt ====
    Resultado Backtracking: 1640690
    Resultado Aproximacion: 1640690
    Diferencia: 0
    Proporcion: 1.0

==== Set 6_4.txt ====
    Resultado Backtracking: 807418
    Resultado Aproximacion: 807418
    Diferencia: 0
    Proporcion: 1.0

==== Set 8_3.txt ====
    Resultado Backtracking: 4298131
    Resultado Aproximacion: 4298131
    Diferencia: 0
    Proporcion: 1.0

==== Set 10_3.txt ====
    Resultado Backtracking: 385249
    Resultado Aproximacion: 385249
    Diferencia: 0
    Proporcion: 1.0

==== Set 10_5.txt ====
    Resultado Backtracking: 355882
    Resultado Aproximacion: 355882
    Diferencia: 0
    Proporcion: 1.0

==== Set 10_10.txt ====
    Resultado Backtracking: 172295
    Resultado Aproximacion: 172295
    Diferencia: 0
    Proporcion: 1.0

==== Set 11_5.txt ====
    Resultado Backtracking: 2906564
    Resultado Aproximacion: 2906564
    Diferencia: 0
    Proporcion: 1.0

==== Set 14_3.txt ====
    Resultado Backtracking: 15659106
    Resultado Aproximacion: 15664276
    Diferencia: 5170
    Proporcion: 1.0003301593334895

```

```
==== Set 14_4.txt ====
    Resultado Backtracking: 15292055
    Resultado Aproximacion: 15292085
    Diferencia: 30
    Proporcion: 1.000001961803041

==== Set 14_6.txt ====
    Resultado Backtracking: 10694510
    Resultado Aproximacion: 10700172
    Diferencia: 5662
    Proporcion: 1.0005294305208934

==== Set 15_4.txt ====
    Resultado Backtracking: 4311889
    Resultado Aproximacion: 4317075
    Diferencia: 5186
    Proporcion: 1.0012027211275616

==== Set 15_6.txt ====
    Resultado Backtracking: 6377225
    Resultado Aproximacion: 6377501
    Diferencia: 276
    Proporcion: 1.0000432790124232

==== Set 17_5.txt ====
    Resultado Backtracking: 15974095
    Resultado Aproximacion: 15975947
    Diferencia: 1852
    Proporcion: 1.0001159377103992

==== Set 17_7.txt ====
    Resultado Backtracking: 11513230
    Resultado Aproximacion: 11513230
    Diferencia: 0
    Proporcion: 1.0

==== Set 17_10.txt ====
    Resultado Backtracking: 5427764
    Resultado Aproximacion: 5430512
    Diferencia: 2748
    Proporcion: 1.0005062858296714

==== Set 18_6.txt ====
    Resultado Backtracking: 10322822
    Resultado Aproximacion: 10325588
    Diferencia: 2766
    Proporcion: 1.0002679499849945

==== Set 18_8.txt ====
    Resultado Backtracking: 11971097
    Resultado Aproximacion: 12000279
    Diferencia: 29182
    Proporcion: 1.002437704748362

==== Set 20_4.txt ====
    Resultado Backtracking: 21081875
    Resultado Aproximacion: 21083935
    Diferencia: 2060
    Proporcion: 1.0000977142687735

==== Set 20_5.txt ====
    Resultado Backtracking: 16828799
    Resultado Aproximacion: 16838539
    Diferencia: 9740
    Proporcion: 1.0005787697624768

==== Set 20_8.txt ====
    Resultado Backtracking: 11417428
    Resultado Aproximacion: 11423826
    Diferencia: 6398
```

```
    Proporcion: 1.0005603713901239

Proporcion del peor caso: 1.002437704748362
```

Podemos observar que, si bien la aproximación no resulta optima en todos los casos, en aquellos en los que backtracking obtiene una solución mejor la diferencia entre ambas resulta muy pequeña. Para los sets provistos por la cátedra, esta aproximación esta acotada por:

$$\frac{S}{S^*} \leq 1{,}0025$$

4.4. Conclusión

El algoritmo de aproximación greedy presentado constituye una solución práctica y eficiente para el problema de asignación de maestros a grupos. Con complejidad temporal $O(n \cdot k)$ en el caso general y $O(n^2)$ en el peor caso (cuando $k = n$), el algoritmo es significativamente más rápido que los enfoques exactos basados en backtracking, cuya complejidad es exponencial.

La estrategia de minimizar localmente el incremento cuadrático en cada asignación resulta en una búsqueda intuitiva que tiende a producir distribuciones balanceadas. Si bien no existen garantías teóricas de factor de aproximación constante en el caso general, esta aproximación ofrece un excelente balance entre calidad de solución y eficiencia computacional, siendo especialmente útil para instancias de gran tamaño donde el backtracking resulta computacionalmente intratable.

La comparación con el backtracking demuestra que, en la mayoría de instancias prácticas, la pérdida de calidad de solución es marginal comparada con la ganancia sustancial en eficiencia, haciendo de la aproximación greedy la opción preferida para aplicaciones reales.

5. Mediciones

Para corroborar la complejidad temporal documentada se generaron conjuntos de datos de tamaños y características diversas, los cuales se utilizaron para tomar mediciones del tiempo de ejecución de los algoritmos propuestos.

En concreto, cada maestro tiene una cantidad de fuerza aleatoria entre 1 y 1,000 en cualquiera de los conjuntos siendo la cantidad de grupos solicitada la variante de cada set, estos son:

- Grupos chicos: Simula la solicitud de muchos grupos con pocos maestros en cada uno, calculando la cantidad de grupos como $\frac{3}{4}n$.
- Grupos medianos: Simula la solicitud de muchos grupos con pocos maestros en cada uno, calculando la cantidad de grupos como $\frac{1}{2}n$.
- Grupos grandes: Simula la solicitud de muchos grupos con pocos maestros en cada uno, calculando la cantidad de grupos como $\frac{1}{4}n$.

Donde n representa la cantidad de maestros.

Se generaron sets de datos para corroborar los tiempos de ejecución de los algoritmos, con largos de: entre 5 y 25 para el algoritmo de backtracking, entre 5 y 13/15 para el algoritmo de programación lineal, y entre 1,000 y 20,000 maestros para el algoritmo de aproximación, resultando inmanejables para los algoritmos anteriores.

Luego, se ajustaron las siguientes curvas a los datos obtenidos mediante el método de cuadrados mínimos:

$$f_1(x) = c_1 x + c_2 \quad f_2(x) = c_1 x^2 + c_2 \quad f_3(x) = c_1 e^{c_2 x}$$

Finalmente, se recopilaron los resultados y se generaron gráficos que muestran la relación entre el tiempo de ejecución y el tamaño de entrada (cantidad de maestros), así como el error de ajuste respecto a las distintas curvas.

A continuación se presentan los resultados obtenidos para cada algoritmo.

5.1. Backtracking

Analizando los resultados obtenidos, se observa que el tiempo de ejecución del algoritmo de backtracking varía significativamente según la cantidad de grupos solicitados, aun cuando la cantidad de maestros sea similar. En particular, los conjuntos con mayor cantidad de grupos (grupos chicos) presentan tiempos de ejecución notablemente menores respecto a los conjuntos con pocos grupos (grupos grandes). Esto se debe a que, al aumentar la cantidad de grupos, la poda por simetría resulta mucho más efectiva, ya que se incrementa la cantidad de configuraciones equivalentes que pueden ser descartadas rápidamente. Esto resulta en que para los sets de grupos medianos y chicos, para los cuales la cantidad de grupos representa una proporción considerable de la cantidad de maestros, el algoritmo se ejecuta de forma practicamente instantanea. Por el contrario, cuando hay pocos grupos en comparación a maestros, la poda pierde efectividad y el espacio de búsqueda se vuelve más difícil de reducir, incrementando el tiempo de ejecución.

Se observa que el ajuste exponencial es el que mejor representa el crecimiento del tiempo de ejecución en la mayoría de los casos, especialmente para los conjuntos con pocos grupos (grupos grandes), coincidiendo con la complejidad esperada.

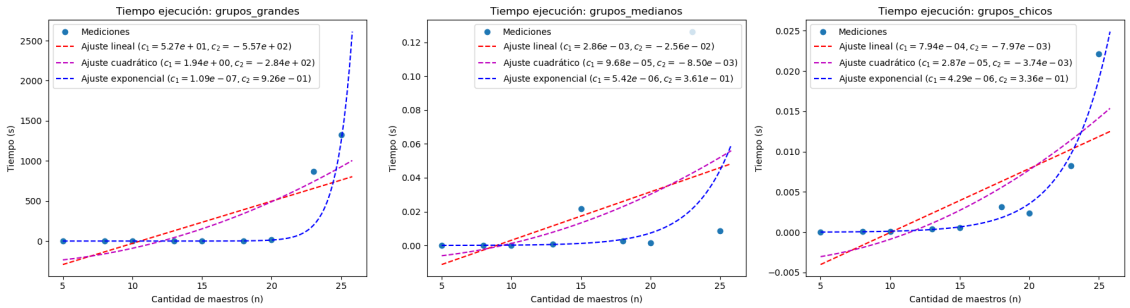

Figura 1: Tiempos de ejecución del algoritmo de Backtracking

Error absoluto:

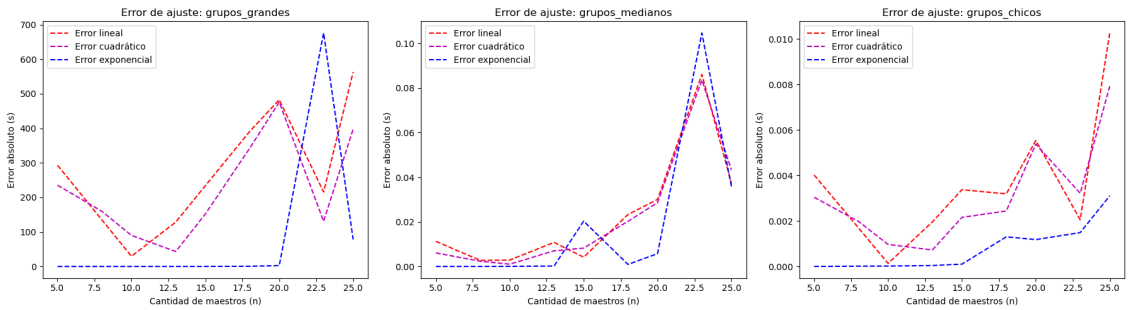

Figura 2: Error absoluto del ajuste

Error cuadrático total:

```
Conjunto grupos_grandes
  Para n: 925120.239677646
  Para n^2: 632705.5355289871
  Para exp: 462890.13836334797

```

```
Conjunto grupos_medianos
  Para n: 0.010519325755714182
  Para n^2: 0.010252864233444958
  Para exp: 0.01268009111087892

Conjunto grupos_chicos
  Para n: 0.00018493207878696546
  Para n^2: 0.00012760764647762795
  Para exp: 1.5025602402769448e-05
```

5.2. Programación Líneal

Al analizar los resultados, se observa que el tiempo de ejecución del algoritmo varía significativamente según la cantidad de grupos solicitados. En particular, los conjuntos con mayor cantidad de grupos (grupos chicos) presentan tiempos de ejecución considerablemente superiores respecto a los conjuntos con menos grupos (grupos grandes). Esto se debe a que, al aumentar la cantidad de grupos, crece polinomialmente la cantidad de variables y restricciones a considerar, incrementando la complejidad del problema. Por el contrario, en los conjuntos con grupos grandes, la menor cantidad de grupos reduce la complejidad y, en consecuencia, el tiempo de ejecución es notablemente menor.

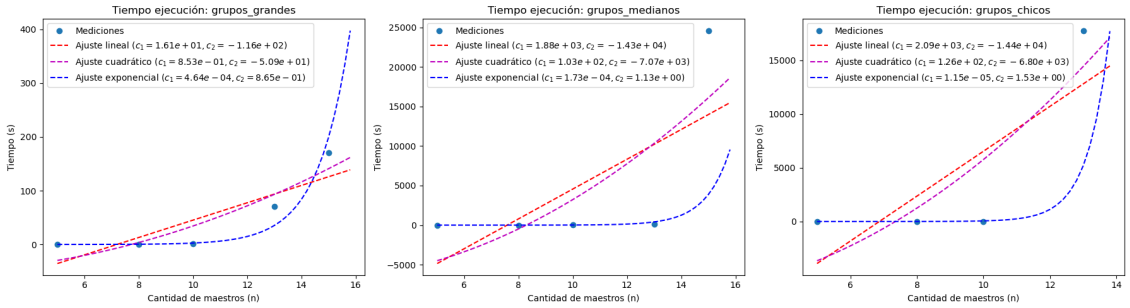

Figura 3: Tiempos de ejecución para el algoritmo por programacion lineal

Error absoluto:

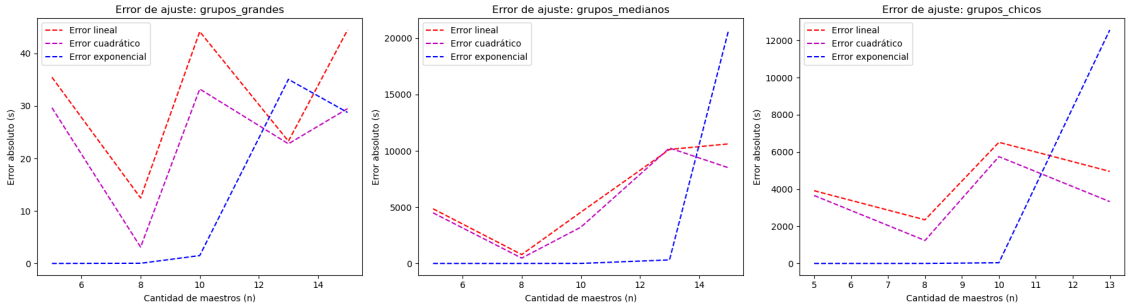

Figura 4: Error absoluto del ajuste

Error cuadrático total:

```
Conjunto grupos_grandes
  Para n: 5879.414691960242
  Para n^2: 3380.2031557125856
  Para exp: 2058.054877318803
```

```

Conjunto grupos_medianos
  Para n: 260387953.16012424
  Para n^2: 208138490.38385913
  Para exp: 429859726.0082914

Conjunto grupos_chicos
  Para n: 87920067.98690179
  Para n^2: 59049456.174902126
  Para exp: 157769703.6188821
```

5.3. Aproximación

Se observa que las mediciones obtenidas corroborar la complejidad teorizada, siendo la función cuadrática la que mejor ajusta los datos obtenidos.

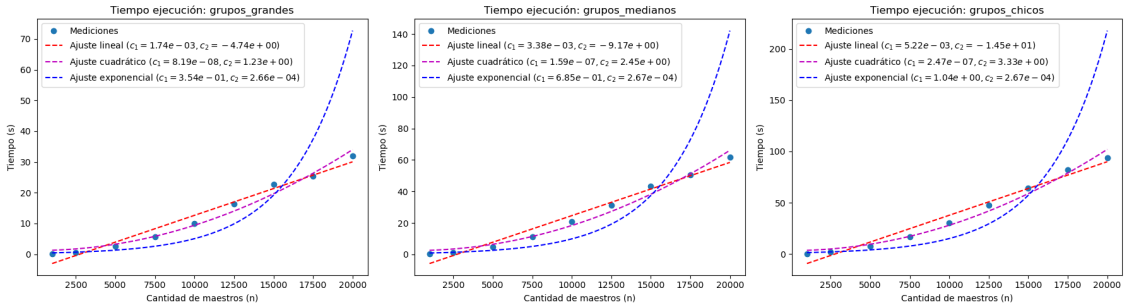

Figura 5: Tiempos de ejecución para el algoritmo de aproximación

Error absoluto:

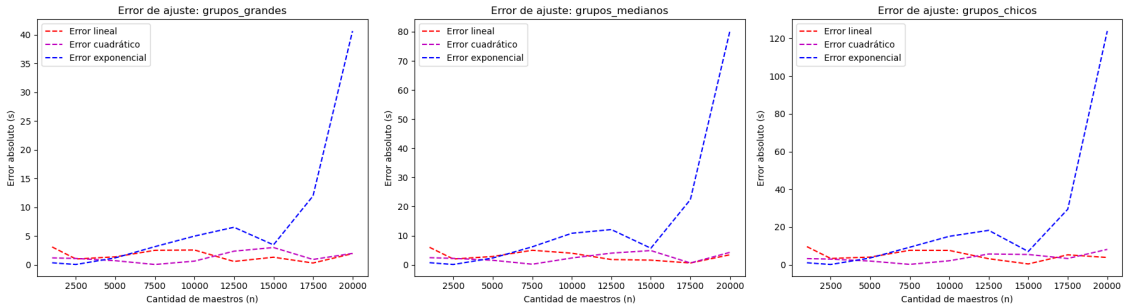

Figura 6: Error absoluto del ajuste

Error cuadrático total:

```
Conjunto grupos_grandes
  Para n: 31.739163949098607
  Para n^2: 23.231124690071294
  Para exp: 1888.21792429484

Conjunto grupos_medianos
  Para n: 105.66695481610046
  Para n^2: 76.28871832372174
  Para exp: 7277.7563655055255

Conjunto grupos_chicos
```

```
Para n: 289.50759759061384
Para n^2: 167.6295605248999
Para exp: 16915.99493382284
```

5.4. Conclusiones

Contrastando los resultados obtenidos, se observa que el algoritmo de backtracking es más viable en tiempo de ejecución que la programación lineal para instancias pequeñas y medianas, especialmente cuando la cantidad de grupos es alta y la poda por simetría puede aprovecharse. Por ejemplo, para 25 maestros en el caso de grupos grandes, el tiempo máximo registrado fue de unos 36 minutos, mientras que para 13 maestros en grupos chicos, la programación lineal requirió hasta 5 horas. Sin embargo, si se requiere manejar una cantidad grande de maestros, backtracking tampoco resulta práctico y es necesario recurrir a una aproximación: con este método, instancias de hasta 20,000 maestros pueden resolverse en aproximadamente 2 minutos.

En definitiva, la elección del algoritmo depende principalmente del tamaño y la estructura del problema; para pocos maestros y muchos grupos, backtracking es una buena opción, mientras que la programación lineal puede ser útil en instancias pequeñas con pocos grupos, pero para instancias grandes la aproximación es indispensable.

A. Resultados sobre sets de la catedra

```
=== ALGORITMO: backtracking ===

Grupo 1: Yakone, Yue, Pakku
Grupo 2: Wei, Pakku I
Coeficiente: 1894340
5_2.txt: OPTIMO

Grupo 1: Wei, Hasook
Grupo 2: Senna, Hama I
Grupo 3: Hama, Wei I
Coeficiente: 1640690
6_3.txt: OPTIMO

Grupo 1: Huu
Grupo 2: Eska
Grupo 3: Sura
Grupo 4: Hama, Sangok, Ming-Hua
Coeficiente: 807418
6_4.txt: OPTIMO

Grupo 1: Katara, Rafa
Grupo 2: Hama, Unalaq, Amon, Tonraq
Grupo 3: Misu, Hasook
Coeficiente: 4298131
8_3.txt: OPTIMO

Grupo 1: Sangok, Yue, Pakku
Grupo 2: Eska, Sura, Tonraq
Grupo 3: Siku, Wei, Amon, La
Coeficiente: 385249
10_3.txt: OPTIMO

Grupo 1: Katara, Hasook
Grupo 2: Misu, Eska
Grupo 3: Siku I, Desna
Grupo 4: Siku, Sura
Grupo 5: Desna I, Unalaq
Coeficiente: 355882
10_5.txt: OPTIMO

Grupo 1: Ming-Hua
Grupo 2: Amon
Grupo 3: Senna
Grupo 4: Tho
Grupo 5: Desna
Grupo 6: Rafa
Grupo 7: Kya
Grupo 8: Sura
Grupo 9: Katara
Grupo 10: Unalaq
Coeficiente: 172295
10_10.txt: OPTIMO

Grupo 1: Yakone
Grupo 2: Siku I
Grupo 3: Tonraq, Siku
Grupo 4: Misu, Hasook, Siku II, Hama
Grupo 5: Misu I, Kya, Pakku
Coeficiente: 2906564
11_5.txt: OPTIMO

Grupo 1: Huu, Kuruk, Tonraq, Eska I, Hama
Grupo 2: Sangok, Rafa, Tonraq I, Eska II
Grupo 3: Sangok I, Misu, Katara, Wei, Eska
Coeficiente: 15659106
14_3.txt: OPTIMO

Grupo 1: Pakku I, Misu, Hasook
```

```
Grupo 2: Tonraq, Huu I, Tho
Grupo 3: La, Ming-Hua, Huu II
Grupo 4: Kya I, Huu, Pakku, Kya, Yue
Coeficiente: 15292055
14_4.txt: OPTIMO

Grupo 1: Ming-Hua, Wei
Grupo 2: Siku, Desna
Grupo 3: Unalaq, Misu
Grupo 4: Amon, Senna
Grupo 5: Misu I, Ming-Hua I
Grupo 6: Kuruk I, Kuruk, La, Kya
Coeficiente: 10694510
14_6.txt: OPTIMO

Grupo 1: Katara, Tonraq I, Yue, Kuruk I
Grupo 2: Kuruk, Tonraq, Desna, Tho
Grupo 3: Hasook, Yakone, Pakku, Kya
Grupo 4: Senna, Wei, Hama
Coeficiente: 4311889
15_4.txt: OPTIMO

Grupo 1: Senna II, Huu I
Grupo 2: Amon, Hasook, Senna I
Grupo 3: Hasook I, Rafa
Grupo 4: Pakku, Senna
Grupo 5: La, Ming-Hua, Sura
Grupo 6: Huu, Tonraq, Wei
Coeficiente: 6377225
15_6.txt: OPTIMO

Grupo 1: Sangok I, Misu I, Rafa
Grupo 2: Yakone I, Yakone, Tho
Grupo 3: Sangok, Huu, Ming-Hua, Misu
Grupo 4: Senna I, Amon I, Senna
Grupo 5: Wei, Unalaq, Amon, Hama
Coeficiente: 15974095
17_5.txt: OPTIMO

Grupo 1: Pakku II, Hasook I, Pakku
Grupo 2: Rafa, Wei I
Grupo 3: Hama, Eska, Desna, Hasook
Grupo 4: Wei, La
Grupo 5: Unalaq, Tonraq
Grupo 6: Pakku I, Yue
Grupo 7: Siku, Sangok
Coeficiente: 11513230
17_7.txt: OPTIMO

Grupo 1: Yue
Grupo 2: Rafa
Grupo 3: Pakku I
Grupo 4: Siku
Grupo 5: Wei I, Eska
Grupo 6: Misu, Pakku
Grupo 7: Sangok, Siku I
Grupo 8: Wei, Tho
Grupo 9: Amon, Yue I, Sura
Grupo 10: Hama, Desna
Coeficiente: 5427764
17_10.txt: OPTIMO

Grupo 1: Misu I, Kya
Grupo 2: Yue II, Kuruk, Huu
Grupo 3: Yue I, Yakone
Grupo 4: Tho, Hama I, Sangok
Grupo 5: Senna, Amon, Kya I, Misu
Grupo 6: Hama, Kuruk I, Pakku, Yue
Coeficiente: 10322822
18_6.txt: OPTIMO
```

```

Grupo 1: Amon II, Senna
Grupo 2: Sura, Hama
Grupo 3: Kya I, Amon I
Grupo 4: Siku, Tho I
Grupo 5: Yakone, Amon
Grupo 6: Rafa I, Desna, Katara
Grupo 7: Rafa, Wei, Ming-Hua
Grupo 8: Tho, Kya
Coeficiente: 11971097
18_8.txt: OPTIMO

Grupo 1: Sura I, Tho, Sura, Sangok I
Grupo 2: Siku I, Wei, Ming-Hua, Sangok II
Grupo 3: Sangok, Yue, Tonraq I, Hama, Katara, Hasook
Grupo 4: Rafa, Tonraq, Tho I, Siku, Kya, Wei I
Coeficiente: 21081875
20_4.txt: OPTIMO

Grupo 1: Huu, Kuruk, Ming-Hua
Grupo 2: Desna, Sura, Tonraq, Misu II
Grupo 3: Yakone, Misu, Huu II, Yue, Huu III
Grupo 4: Huu I, Siku, Wei
Grupo 5: Hasook I, Hasook, Tonraq I, Yue I, Misu I
Coeficiente: 16828799
20_5.txt: OPTIMO

Grupo 1: Wei, Sangok, Hama I
Grupo 2: Kuruk, Ming-Hua, Katara
Grupo 3: Unalaq, Amon I
Grupo 4: Yakone, Sura, Sura I
Grupo 5: Tonraq, Pakku, La
Grupo 6: Misu, Ming-Hua I
Grupo 7: Tho, Amon
Grupo 8: Hama, Rafa
Coeficiente: 11417428
20_8.txt: OPTIMO

=== ALGORITMO: programacion_lineal ===

Grupo 1: Pakku I, Wei
Grupo 2: Pakku, Yue, Yakone
Coeficiente: 1894340
5_2.txt: OPTIMO

Grupo 1: Senna, Hama I
Grupo 2: Hasook, Wei
Grupo 3: Hama, Wei I
Coeficiente: 1640690
6_3.txt: OPTIMO

Grupo 1: Huu
Grupo 2: Hama, Sangok, Ming-Hua
Grupo 3: Sura
Grupo 4: Eska
Coeficiente: 807418
6_4.txt: OPTIMO

Grupo 1: Unalaq, Hama, Tonraq, Amon
Grupo 2: Katara, Rafa
Grupo 3: Misu, Hasook
Coeficiente: 4298131
8_3.txt: OPTIMO

Grupo 1: Wei, Siku, Amon, La
Grupo 2: Sangok, Yue, Pakku
Grupo 3: Eska, Sura, Tonraq
Coeficiente: 385249
10_3.txt: OPTIMO

```

```
Grupo 1: Hasook, Katara
Grupo 2: Siku, Sura
Grupo 3: Desna, Siku I
Grupo 4: Misu, Eska
Grupo 5: Desna I, Unalaq
Coeficiente: 355882
10_5.txt: OPTIMO

Grupo 1: Desna
Grupo 2: Amon
Grupo 3: Senna
Grupo 4: Rafa
Grupo 5: Katara
Grupo 6: Kya
Grupo 7: Sura
Grupo 8: Unalaq
Grupo 9: Tho
Grupo 10: Ming-Hua
Coeficiente: 172295
10_10.txt: OPTIMO

Grupo 1: Hasook, Misu, Hama, Siku II
Grupo 2: Pakku, Misu I, Kya
Grupo 3: Yakone
Grupo 4: Siku, Tonraq
Grupo 5: Siku I
Coeficiente: 2906564
11_5.txt: OPTIMO

Grupo 1: Huu, Tonraq, Kuruk, Eska I, Hama
Grupo 2: Eska, Misu, Katara, Wei, Sangok I
Grupo 3: Sangok, Eska II, Tonraq I, Rafa
Coeficiente: 15659106
14_3.txt: OPTIMO
```

B. Resultados de las mediciones

```
=== Conjunto grupos_grandes ===

Algoritmo programacion_lineal
  Cantidad de maestros:        5 Tiempo promedio: 0.043514
  Cantidad de maestros:        8 Tiempo promedio: 0.517770
  Cantidad de maestros:       10 Tiempo promedio: 1.127610
  Cantidad de maestros:       13 Tiempo promedio: 70.387524
  Cantidad de maestros:       15 Tiempo promedio: 170.398961

Algoritmo backtracking
  Cantidad de maestros:        5 Tiempo promedio: 0.000082
  Cantidad de maestros:        8 Tiempo promedio: 0.000148
  Cantidad de maestros:       10 Tiempo promedio: 0.000274
  Cantidad de maestros:       13 Tiempo promedio: 0.029350
  Cantidad de maestros:       15 Tiempo promedio: 0.013927
  Cantidad de maestros:       18 Tiempo promedio: 1.238598
  Cantidad de maestros:       20 Tiempo promedio: 14.761565
  Cantidad de maestros:       23 Tiempo promedio: 871.167746
  Cantidad de maestros:       25 Tiempo promedio: 1323.711758

Algoritmo aproximacion
  Cantidad de maestros:     1000 Tiempo promedio: 0.103060
  Cantidad de maestros:     2500 Tiempo promedio: 0.626610
  Cantidad de maestros:     5000 Tiempo promedio: 2.558995
  Cantidad de maestros:     7500 Tiempo promedio: 5.776449
  Cantidad de maestros:    10000 Tiempo promedio: 10.069347
  Cantidad de maestros:    12500 Tiempo promedio: 16.406918
  Cantidad de maestros:    15000 Tiempo promedio: 22.675041
  Cantidad de maestros:    17500 Tiempo promedio: 25.377974
  Cantidad de maestros:    20000 Tiempo promedio: 32.004493
```

```

=== Conjunto grupos_medianos ===

Algoritmo programacion_lineal
  Cantidad de maestros:        5 Tiempo promedio: 0.176346
  Cantidad de maestros:        8 Tiempo promedio: 0.587869
  Cantidad de maestros:       10 Tiempo promedio: 6.843057
  Cantidad de maestros:       13 Tiempo promedio: 87.158636
  Cantidad de maestros:       15 Tiempo promedio: 24605.286894

Algoritmo backtracking
  Cantidad de maestros:        5 Tiempo promedio: 0.000016
  Cantidad de maestros:        8 Tiempo promedio: 0.000071
  Cantidad de maestros:       10 Tiempo promedio: 0.000127
  Cantidad de maestros:       13 Tiempo promedio: 0.000808
  Cantidad de maestros:       15 Tiempo promedio: 0.021539
  Cantidad de maestros:       18 Tiempo promedio: 0.002600
  Cantidad de maestros:       20 Tiempo promedio: 0.001556
  Cantidad de maestros:       23 Tiempo promedio: 0.126281
  Cantidad de maestros:       25 Tiempo promedio: 0.008707

Algoritmo aproximacion
  Cantidad de maestros:     1000 Tiempo promedio: 0.192670
  Cantidad de maestros:     2500 Tiempo promedio: 1.252295
  Cantidad de maestros:     5000 Tiempo promedio: 4.898560
  Cantidad de maestros:     7500 Tiempo promedio: 11.221121
  Cantidad de maestros:    10000 Tiempo promedio: 20.673421
  Cantidad de maestros:    12500 Tiempo promedio: 31.295592
  Cantidad de maestros:    15000 Tiempo promedio: 43.113042
  Cantidad de maestros:    17500 Tiempo promedio: 50.577185
  Cantidad de maestros:    20000 Tiempo promedio: 61.847002

=== Conjunto grupos_chicos ===

Algoritmo programacion_lineal
  Cantidad de maestros:        5 Tiempo promedio: 0.065959
  Cantidad de maestros:        8 Tiempo promedio: 1.290368
  Cantidad de maestros:       10 Tiempo promedio: 10.690520
  Cantidad de maestros:       13 Tiempo promedio: 17753.318745

Algoritmo backtracking
  Cantidad de maestros:        5 Tiempo promedio: 0.000021
  Cantidad de maestros:        8 Tiempo promedio: 0.000079
  Cantidad de maestros:       10 Tiempo promedio: 0.000102
  Cantidad de maestros:       13 Tiempo promedio: 0.000384
  Cantidad de maestros:       15 Tiempo promedio: 0.000556
  Cantidad de maestros:       18 Tiempo promedio: 0.003118
  Cantidad de maestros:       20 Tiempo promedio: 0.002363
  Cantidad de maestros:       23 Tiempo promedio: 0.008227
  Cantidad de maestros:       25 Tiempo promedio: 0.022139

Algoritmo aproximacion
  Cantidad de maestros:     1000 Tiempo promedio: 0.299881
  Cantidad de maestros:     2500 Tiempo promedio: 1.883521
  Cantidad de maestros:     5000 Tiempo promedio: 7.538921
  Cantidad de maestros:     7500 Tiempo promedio: 16.992157
  Cantidad de maestros:    10000 Tiempo promedio: 30.099835
  Cantidad de maestros:    12500 Tiempo promedio: 47.559922
  Cantidad de maestros:    15000 Tiempo promedio: 64.244841
  Cantidad de maestros:    17500 Tiempo promedio: 82.155894
  Cantidad de maestros:    20000 Tiempo promedio: 93.798903
```

C. Código de generación de datos

```
import random
import sys

```

```python
MODO_ESCRITURA = 'w'
ACLARACION = "La primera linea indica la cantidad de grupos a formar, las siguientes son de la forma 'nombre maestro, habilidad'.\n"
GRUPOS_GRANDES = "grupos_grandes"  # Pocos grupos, muchos maestros por grupo
GRUPOS_MEDIANOS = "grupos_medianos"  # Grupos medianos
GRUPOS_CHICOS = "grupos_chicos"  # Muchos grupos, pocos maestros por grupo
NOMBRES = [
  "Ming-Hua", "Yakone", "Tho", "Rafa", "Unalaq", "Wei", "Katara", "Hama",
  "Sura", "Pakku", "La", "Amon", "Ming-Hua I", "Amon I", "Tonraq",
  "Sangok", "Kuruk", "Hama I", "Sura I", "Misu"
]

def generador_generico() -> tuple[str, int]:
  nombre = random.choice(NOMBRES)
  fuerza = random.randint(1, 1000)
  return nombre, fuerza

def escribir_datos(cantidad_datos, ruta, callback_generador, tipo_grupos=None) -> None:
  with open(ruta, MODO_ESCRITURA) as archivo:
    archivo.write(ACLARACION)
    if tipo_grupos == GRUPOS_GRANDES:
      cant_grupos = max(2, cantidad_datos // 4) # 1/4 de maestros
    elif tipo_grupos == GRUPOS_MEDIANOS:
      cant_grupos = max(2, cantidad_datos // 2) # 2/4 de maestros
    elif tipo_grupos == GRUPOS_CHICOS:
      cant_grupos = max(2, (3 * cantidad_datos) // 4) # 3/4 de maestros
    else:
      cant_grupos = random.randint(2, cantidad_datos // 2)

    archivo.write(f"{cant_grupos}\n")
    for _ in range(cantidad_datos):
      nombre, fuerza = callback_generador()
      archivo.write(f"{nombre}, {fuerza}\n")

def generar(cantidad_datos, ruta_salida, generador):
  GENERADORES = {
      GRUPOS_GRANDES: generador_generico,
      GRUPOS_MEDIANOS: generador_generico,
      GRUPOS_CHICOS: generador_generico
  }

  if generador not in GENERADORES:
    print(f"Tipo de generador '{generador}' no reconocido. Opciones: {list(GENERADORES.keys())}")
    sys.exit(1)

  callback = GENERADORES[generador]
  escribir_datos(int(cantidad_datos), ruta_salida, callback, generador)
```